



微算機系統實習 MICROPROCESSOR SYSTEMS LAB. AUTUMN, 2026

Instructor : Yen-Lin Chen(陳彥霖), Ph.D.

Professor

Dept. Computer Science and Information Engineering
National Taipei University of Technology



作業評分方式

- 實驗部分佔該次實驗的總比例70%
小組報告總分佔該次實驗的總比例15%
個人報告總分佔該次實驗的總比例15%
*當次實驗會因為難易度不同佔學期總成績的實驗分數比例也不同
- **實驗報告上傳格式，公告於i學園**
- **實驗報告以小組為單位繳交報告**

請同學上傳報告時依照上面的格式上傳：

- 一、組別與組員名單
- 二、實驗步驟截圖與說明
- 三、組員貢獻比例（組員%數加總必須等於100%）
- 四、個人心得（額外單獨繳交）

注意事項：

- 實驗報告單獨分數為100分，以上第一、三、四點每缺少一項報告分數扣20分。第二點缺少報告扣40分。另外第二、四點不完整會依照狀況扣分，最高扣到該項目的上限分數。
- 貢獻比例分配方式為：報告總分*人數*組員貢獻比例=組員報告得分



作業繳交

- 基本繳交時間
 - 實驗、報告：
 - 實驗驗收: 公布實驗後的下個星期三(9/23)下課結束前
 - 報告: 實驗驗收後的下個星期三(9/30)結束前(23:59)
 - * 若有因為特殊原因繳交時間有變動助教會另外公布
 - 超過時間遲交每隔一週(含一週內) **分數打8折**，採累計連乘方式，實驗與報告打折是分開算的，
 - 最多遲交兩週，〈遲交的項目單獨分數〉 * 0.64**
 - 舉例：
 - 遲交三天——以遲交一週計算 $\langle \text{遲交的項目單獨分數} \rangle * 0.8 = \text{該項目得到的分數}$
 - 遲交九天——以遲交兩週計算 $\langle \text{遲交的項目單獨分數} \rangle * 0.8 * 0.8 = \text{該項目得到的分數}$
- 以上配分與注意事項有問題請聯絡助教



小組報告格式規定

- 實驗目的
該段落須回答為什麼要做這個實驗，以及想解決或探討什麼具體問題
- 設備與零件
實驗中用到的設備、器材以及工作環境
- 工作原理與預期結果
包含實驗過程與預期結果
- 實習結果與討論
程式碼之截圖並加以說明
對成果的照片或影片進行描述，若有影片則須提供連結
遇到的問題&問題的解決方式
- 分工
說明小組成員分工內容與比例。



個人報告內容

- 實驗心得：（150字）
 - 此次實驗的目的？
 - 在實驗上遇到的問題？怎麼解決？
 - 實驗結果跟你預期的內容是否一致？
- 組員貢獻度及工作內容：
 - 陳大明：33%，資料查詢、文書處理、實驗實作。
 - 陳忠義：33%，實驗設計、程式規劃、測試與除錯。
 - 陳小美：33%，小組報告、買飲料、買午餐。



作業繳交格式

- 請確實依照規定的檔案名稱、副檔名上傳，**否則不予計分**
- 小組、個人報告檔案格式：
 - 報告本身請使用 **.pdf** 檔
 - 小組：使用”Lab<number>_第<number>組_小組報告”命名。
 - 個人：使用”Lab<number>_第<number>組_<學號>_<姓名>_個人報告”命名。
 - 資料夾請使用”Lab<number>_第<number>組”命名，並壓縮為 **.zip** 檔
 - <number>部分請填零到兩位數，ex: Lab01 、Lab1 
 - 母資料夾裡要包含如下子資料夾
 - -Code //存放專案程式碼
 - -Report //存放報告
 - -若需要拍攝影片，請上傳至雲端空間(ex:Youtube)，附上連結網址、影片截圖至小組報告內。請確保連結網址可以直接觀看影片。
 - -文字說明檔



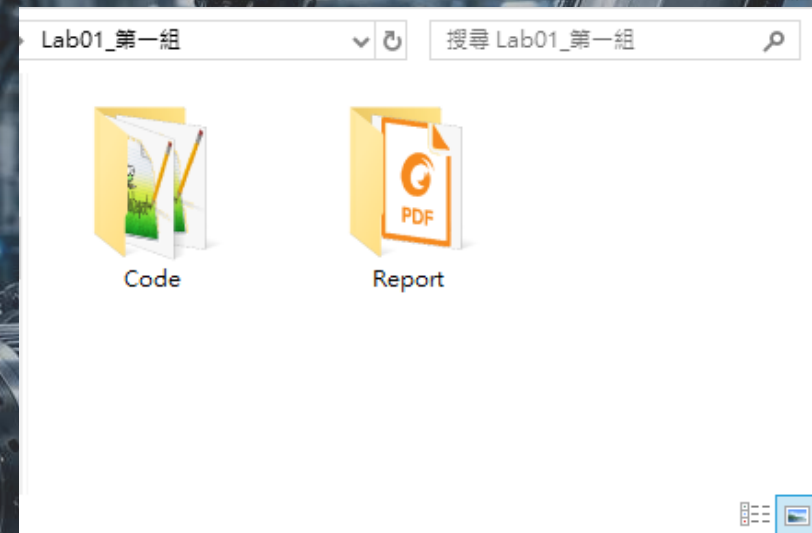
作業繳交格式 cont.

- 小組報告繳交範例：

```
.├─ Lab01_第01組.zip├─ Lab01_第01組├─ Code├─ main.cpp├─ test.cpp├─ Report└─ Lab01_第01組_小組報告.pdf
```

- 個人報告繳交範例：

```
.└─ Lab01_第01組_111ABC001_陳小明_個人報告.pdf
```





說明

- 請依組別與助教領取TX2
 - ✓ 領取TX2時，**須登記MAC**
 - ✓ 該台TX2即為本學期組員共用機台
 - ✓ 每次實作機台為該組固定使用
 - ✓ 請好好愛惜機台
- 若板子有任何問題請通知助教
 - ✓ 不可自行燒錄TX2作業系統及Kernel



本次實驗目標

- 學習如何使用SSH與其它機台進行連線
- 學習如何撰寫Makefile編譯程式
- 學習如何使用跨平台編譯開發嵌入式系統程式

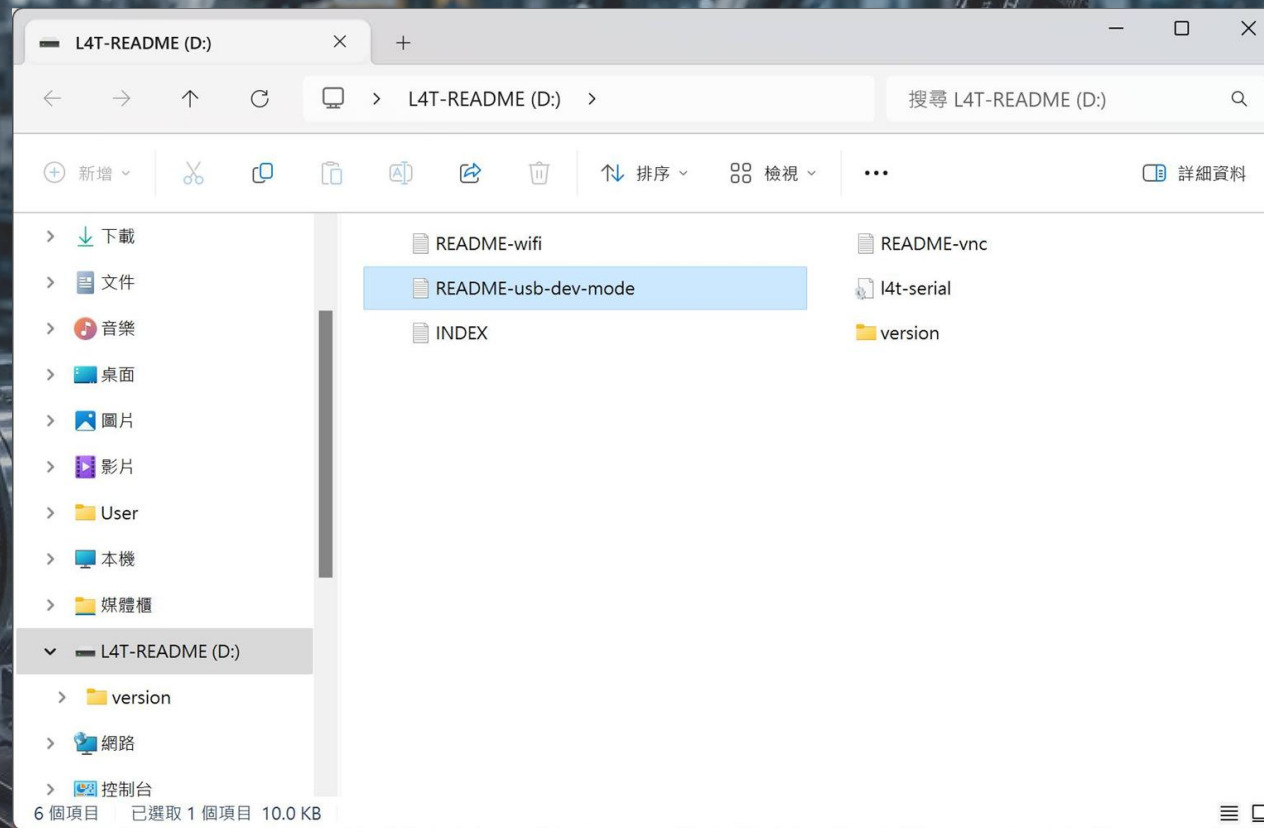
實驗一

跨平台連線教學



連接microUSB接口：確認辨識裝置

- 先確認目前電腦是否辨識裝置，打開README-usb-dev-mode.txt。





查詢IP

- 在README-usb-dev-mode.txt 找到它的IP。
- `ssh nvidia@192.168.55.1`

檔案 編輯 檢視

Jetson.

If you connect multiple Jetson devices to the same host machine, each Jetson device uses the same IPv4 address. This prevents IPv4-based communication with all but one Jetson device, since your host operating system determines which Jetson device it communicates with. To solve this, edit the Jetson-based script that sets up the network and assign a unique network address to each Jetson device. IPv6-based communication works with any number of Jetson devices without modifying the aforementioned script.

Once the Ethernet device is configured, use SSH to connect to the Jetson device.

SSH is natively available on Linux and Macintosh operating systems. Connect using IPv6 with the following command:

```
ssh nvidia@fe80::1%usb0
```

Connect using IPv4 with the following command:

```
ssh nvidia@192.168.55.1
```

On Windows, use the PuTTY application.

Ethernet on Linux

Two USB Ethernet devices are created and the required drivers are automatically activated. These devices are typically named `usb0` and `usb1`, depending on how your Linux distribution is configured, and what other hardware is attached to the system. To avoid the potential for interference between the two USB Ethernet devices, it is best, but not strictly required, to use your host's network configuration tool (e.g. Network Manager), to disable one of the two USB Ethernet interfaces implemented by the Jetson device.



連線TX2

- 在terminal或CMD執行ssh nvidia@192.168.55.1連線TX2
- 若有Host key verification failed的錯誤
 - 請使用ssh-keygen -R 192.168.55.1命令重置金鑰，再連線

```
nvidia@nvidia-desktop: ~  
C:\Users\User>ssh -Y nvidia@192.168.55.1  
nvidia@192.168.55.1's password:  
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.9.201-tegra aarch64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
464 packages can be updated.  
378 updates are security updates.  
  
Last login: Tue Feb 27 01:31:17 2024 from 192.168.55.100  
nvidia@nvidia-desktop:~$
```

```
nvidia@nvidia-desktop: ~  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
VirtualBox:~$ ssh -Y nvidia@192.168.55.1  
The authenticity of host '192.168.55.1 (192.168.55.1)' can't be established.  
ECDSA key fingerprint is SHA256:LRzHamJI4I35HfhtZ7flpEjbeYRmkHSi4SikIjnWthU.  
Are you sure you want to continue connecting (yes/no)? yes  
Warning: Permanently added '192.168.55.1' (ECDSA) to the list of known hosts.  
nvidia@192.168.55.1's password:  
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.9.201-tegra aarch64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
464 packages can be updated.  
378 updates are security updates.  
  
Last login: Tue Feb 27 01:32:09 2024 from 192.168.55.100  
nvidia@nvidia-desktop:~$
```

運用MAKEFILE編譯程式專案



MAKEFILE – 自動化變數

- `$@`: 工作目標檔名。
- `$<`: 第一個被依賴文件的檔名。
- `$$`: 所有被依賴文件的檔名，並以空格隔開這些檔名。
- `$?` : 被依賴文件中有更改過的所有檔名。

```
#用「井」號表明注釋。  
target (要生成的文件): dependencies (被依賴的文件)  
#命令前面用的是「tab」而非空格。誤用空格是初學者容易犯的錯誤。  
命令1  
命令2  
命令3  
.  
.  
命令n  
#可以使用「\」表示續行。注意，「\」之後不能有空格。
```



MAKEFILE – 變數(巨集)

- CXX : C++編譯器的名字，預設值是g++。
- CXXFLAGS : C++編譯器的選項，沒有定義。
- 例如:OBJ = hellofuc.o hellomake.o
- 使用時在前面加 \$() 的符號，如:\$(OBJ)

```
CXX=g++  
CXXFLAGS=-std=c++11  
OBJ=hellofuc.o hellomake.o  
SRC=hellofuc.cpp hellomake.cpp  
EXE=hellomake #filename  
all:$(EXE)  
$(EXE):$(OBJ)  
        $(CXX) -o $(EXE) $(OBJ)  
$(OBJ):$(SRC)  
        $(CXX) -c $(SRC)  
clean:  
        rm -f $(EXE)  
        rm -f $(OBJ)
```

本次實驗需使用C++11的
library，需加此部分

◆ Example:

- CROSS_COMPILE = aarch64-linux-gnu- (TX2是使用ARMv8指令集)
- CXX = \$(CROSS_COMPILE)g++ /* aarch64-linux-gnu-g++ */
- LD = \$(CROSS_COMPILE)ld /*aarch64-linux-gnu-ld */

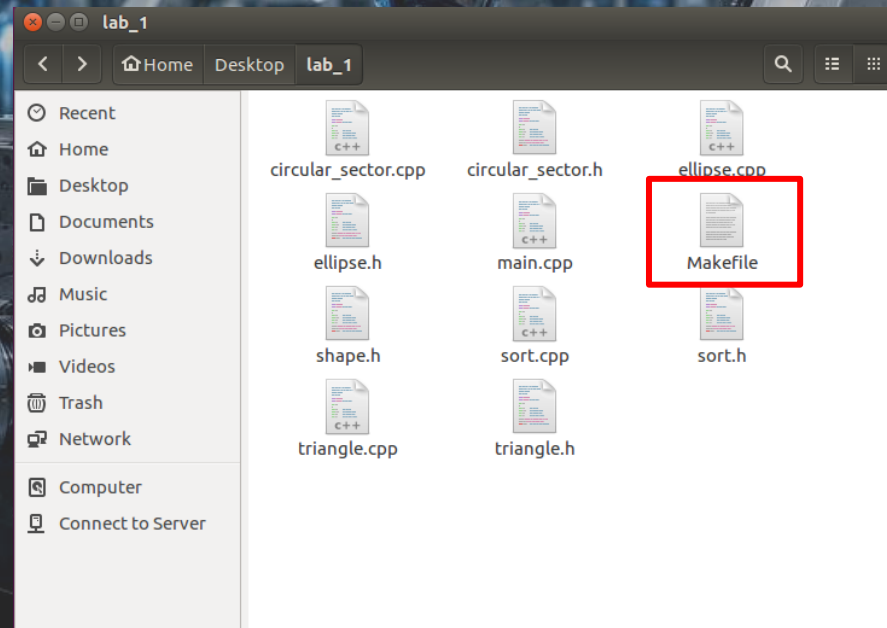
跨平台嵌入式程式開發



實驗一 SOURCE CODE

- 請同學從雲端下載lab1_source_code附件檔

https://drive.google.com/drive/folders/1Ywev6PyLS7s1W_UtmpFZLUWIA1n3sTqk?usp=sharing



- 程式碼內容不需更動，僅需自己編寫Makefile，讓程式可以成功編譯與執行。



實驗一 範例程式說明

- 本次實驗提供下列檔案
- `circular_sector`(扇形)、`ellipse`(橢圓形)及`triangle`(三角形)等種形狀的`.cpp`檔和`.h`檔，程式內容為計算各形狀的周長及面積
- `sort`檔中，透過各形狀依照面積進行升冪排列
- `main`檔中設定各形狀所需的參數，輸出的結果會將各形狀依照面積升冪排列
- 透過本次實驗多種形狀面積的排序，學習如何編寫`Makefile`檔，並完成跨平台編譯。



實驗一 要求

- 項目一：Makefile要有target: all (20%)
 - 編譯出跨平台可執行檔
- 項目二：Makefile要有target: clean (20%)
 - 刪除該執行檔與所有.o檔
- 項目三：在 Makefile 中使用變數，可參考16頁 (10%)
- 項目四：在 Makefile 裡須加入 \$@ (5%)，\$< or \$? (5%)
- 項目五：將虛擬機CROSS_COMPILE後的執行檔傳至TX2，且以ssh連到TX2執行 (10%)
- 項目六：實驗報告與Makefile檔 (30%)



MAKEFILE 範例

```
all:hellofuc.o hellomake.o
    g++ -o hellomake hellofuc.o hellomake.o
hellofuc.o:hellofuc.cpp
    g++ -c -o hellofuc.o hellofuc.cpp
hellomake.o:hellomake.cpp
    g++ -c -o hellomake.o hellomake.cpp
clean:
    rm -f hellomake
    rm -f hellomake.o
    rm -f hellofuc.o
```



實驗一 執行結果

- 將編譯的跨平台執行檔傳輸至TX2(可利用scp傳輸)
- 在終端機(利用SSH遠端進TX2介面)輸入ls，查看home目錄下的檔案，若tx2_exe有出現，代表傳送成功
- 在終端機(TX2)執行tx2_exe，如果出現如下畫面，代表完成跨平

台編譯

```
nvidia@tegra-ubuntu:~$ ls
Desktop      NVIDIA_CUDA-9.0_Samples  Templates
Documents    opencv-2.4.9             tx2_exe
Downloads    Pictures                  Videos
examples.desktop  Public                  VisionWorks-SFM-0.90-Samples
jetson_clocks.sh  tegra_multimedia_api     weston.ini
Music         tegrastats
```

在TX2上的終端機

```
nvidia@tegra-ubuntu:~$ ./tx2_exe
原本的: 62.8319 78.5398 52.3599 50
排序後: 50 52.3599 62.8319 78.5398
作業一完成
```

← 輸出結果如此處